

😊 Perfect. Let's keep going.

7.4.10 — `getCurrentUser()` — centralizing authentication

Right now, imagine every protected server-side function has to repeat:

```
const cookieStore = await cookies();  
  
const sessionId = cookieStore.get("sessionId")?.value;
```

```
const session = await db.session.findUnique({
  where: { id: sessionId },
});
```

That would get ugly **very quickly**.

Instead, we create one server-side helper:

```
// lib/auth.ts

import { cookies } from "next/headers";

export async function getCurrentUser() {
  const cookieStore = await cookies();

  const sessionId = cookieStore.get("sessionId")?.value;

  if (!sessionId) {
    return null;
  }

  const session = await db.session.findUnique({
    where: {
      id: sessionId,
    },
    include: {
      user: true,
    },
  });

  if (!session) {
    return null;
  }

  if (session.expiresAt < new Date()) {
    return null;
  }

  return session.user;
}
```

Now any server-side code can simply do:

```
const user = await getCurrentUser();
```

Why is this useful?

Imagine `/dashboard` :

```
export default async function DashboardPage() {  
  const user = await getCurrentUser();  
  
  if (!user) {  
    redirect("/login");  
  }  
  
  return <h1>Welcome {user.name}</h1>;  
}
```

And `/settings` :

```
export default async function SettingsPage() {  
  const user = await getCurrentUser();  
  
  if (!user) {  
    redirect("/login");  
  }  
  
  return <Settings user={user} />;  
}
```

And an authenticated Server Action:

```
"use server";  
  
export async function updateProfile(formData: FormData) {  
  const user = await getCurrentUser();  
  
  if (!user) {  
    throw new Error("Unauthorized");  
  }  
  
  // update user profile  
}
```

Same authentication logic, one place.



But here's an important distinction

`getCurrentUser()` answers:

Who is the current authenticated user?

It does **NOT** automatically answer:

Is this user allowed to perform this particular operation?

That's authorization.

For example:

```
const user = await getCurrentUser();

if (!user) {
  throw new Error("Unauthorized");
}
```

means:

Are you logged in?



But:

```
if (user.role !== "ADMIN") {
  throw new Error("Forbidden");
}
```

means:

Are you allowed to do this?



So:

```
Authentication
  ↓
Who are you?
  ↓
getCurrentUser()

Authorization
```



↓
What are you allowed to do?
↓
role / ownership / permissions

7.4.11 — 401 vs 403

This is another interview favorite. 🎯

401 Unauthorized

Usually means:

You aren't authenticated.

Example:

No valid session cookie
↓
401



403 Forbidden

Usually means:

You're authenticated, but you're not allowed to perform this action.

Example:

User is logged in
↓
role = USER
↓
trying to access ADMIN resource
↓
403



Think:

401 → "Who are you?"



403 → "I know who you are, but you can't do this."

7.4.12 — One more production-realistic improvement

Earlier we used:

```
crypto.randomUUID()
```

to make the session ID.

The **production principle** is more important than the exact API:

Session IDs must be generated using a cryptographically secure random source and have enough entropy to make guessing infeasible.

You don't want:

```
sessionId = 12345
```



because an attacker could try:

```
12344  
12345  
12346  
...
```



Instead, you want something unpredictable:

```
8f5d9e3a-7c...
```



And the session should also have:

```
expiration  
revocation  
secure cookie settings
```



7.4.13 — What happens when the user logs out?

This completes the lifecycle.

Suppose:

```
Browser cookie  
↓  
sessionId = ABC123
```



Database

↓

ABC123 → user 42

User clicks:

Logout

Server can:

1. Delete/revoke the session

```
await db.session.delete({
  where: {
    id: sessionId,
  },
});
```

2. Remove the cookie

```
cookieStore.delete("sessionId");
```

Now:

Browser

↓

No session cookie

Database

↓

No valid session



Next request:

GET /dashboard



Server:

No session

↓

Not authenticated

↓

redirect("/login")





So we've now completed the entire login lifecycle

Signup

```
password
  ↓
Argon2id
  ↓
password hash
  ↓
DB
```



Login

```
email + password
  ↓
find user
  ↓
verify password hash
  ↓
authentication succeeds
  ↓
create session
  ↓
store session → userId
  ↓
set HttpOnly cookie
```



Subsequent request

```
cookie
  ↓
session ID
  ↓
session lookup
  ↓
user ID
  ↓
current user
```



Authorization

```
current user
  ↓
permissions / role / ownership
```



↓
allowed?

Logout

delete/revoke session
+
delete cookie
↓
authenticated session ends



★ The interview mental model

If they ask you:

"How does session-based authentication work in Next.js?"

Think:

```
LOGIN
↓
email + password
↓
Server
↓
verify Argon2id
↓
User found
↓
Create session
↓
sessionId → userId
↓
HttpOnly session cookie
↓
BROWSER
↓
Future request
↓
cookie sent
↓
SERVER
↓
lookup session
↓
identify user
↓
```



authorization
↓
protected data

🔥 That is 7.4.

Now we're ready for 7.5 — **Server Components + Auth**, where things get especially interesting because **Server Components can directly read the authenticated session on the server without exposing the session logic to the browser.**



okay and before that i have a doubt can we directly contact db from frontend code is it safe and if it is then why we have backend

Yes — **technically, a frontend can directly communicate with some databases**, but for a normal web application, **you generally should not expose your database directly to browser code.**

And this question is actually **the reason backend exists.** 😊 Let's build it from first principles.

1. What would "frontend directly talks to DB" mean?

Normally we have:

Browser
↓
Backend
↓
Database



But imagine:

Browser
↓
Database



For example, your React code somehow does:

```
</> TypeScript
```



```
const users = await database.users.findMany();
```

The browser would need some way to connect to that database.

And that's where the problems begin.

2. Problem #1 — You can't put DB credentials in frontend code

Suppose your database requires:

```
DB_HOST = mydb.com  
DB_USER = admin  
DB_PASSWORD = secret123
```



If you put this in frontend JavaScript:

```
</> TypeScript
```



```
const dbPassword = "secret123";
```

the browser receives that code.

Anyone can open:

DevTools → Sources

and inspect your JavaScript.

So:

```
DB password  
  ↓  
Frontend bundle  
  ↓  
Browser  
  ↓  
Anyone can inspect it ❌
```



You cannot keep a database secret inside browser code.

This is exactly the same reason we don't put:

```
DATABASE_PASSWORD
API_SECRET
PRIVATE_KEY
```



in client-side code.

3. "But what if I hide it in an environment variable?"

This is a VERY common confusion with Next.js.

You might write:

```
</> env

DATABASE_URL=postgres://user:password@...
```



and then:

```
</> TypeScript

process.env.DATABASE_URL
```



That is safe **only if the code stays on the server**.

For example:

```
</> TypeScript

// Server Component
const users = await db.user.findMany();
```

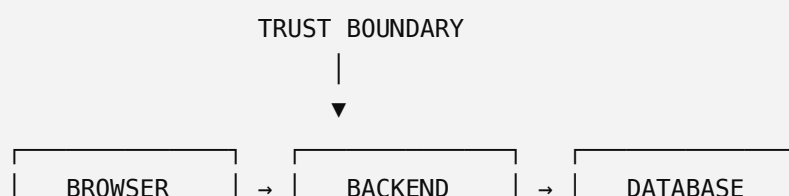


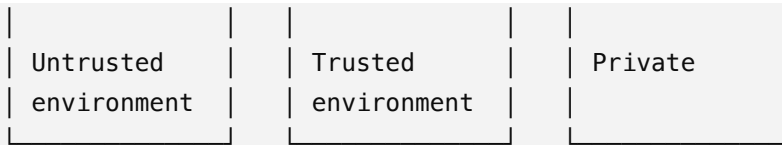
The DB URL remains on the server.

But if you expose it to client-side code, you've lost the secret.

4. So why do we have a backend?

This is the big picture:





The browser belongs to the **user**.

That means you should assume the user can:

- modify JavaScript
- modify requests
- inspect network requests
- call endpoints manually
- change form values
- bypass your UI
- use Postman/curl
- manipulate DevTools

So the browser is **not a trusted security boundary**.

5. Backend is the security gatekeeper

Imagine your app has:

```
DELETE /api/users/42
```



Your frontend might have:

```
</> TypeScript
```



```
<button>Delete Account</button>
```

You might hide the button from normal users:

```
</> TypeScript
```



```
if (user.role === "ADMIN") {  
  return <DeleteButton />;  
}
```

But that is **not security**.

Why?

Because the user can manually call:

```
</> http
```



```
DELETE /api/users/42
```

without clicking your button.

Therefore the backend must check:

Request

↓

Backend

↓

Is user authenticated?

↓

Is user authorized?

↓

Can this user delete user 42?

↓

YES → DB operation

NO → reject



This is why we said earlier:

Authorization must be enforced on the server.

6. Backend also protects your business logic

Imagine your shopping app has:

Product price = ₹10,000



Your frontend displays:

₹10,000



But a malicious user changes their browser request to:

```
</> JSON
```



```
{  
  "productId": 123,  
  "price": 1  
}
```

If your backend blindly trusts the frontend:

Browser



"price = ₹1"



Database



Instead:

Browser



Backend



Fetch actual product price from DB



Validate order



Calculate total



DB



The backend is the **source of truth**.

7. Backend also protects secrets

Suppose you're integrating with Stripe, OpenAI, AWS, etc.

You may have:

STRIPE_SECRET_KEY

OPENAI_API_KEY

AWS_SECRET_ACCESS_KEY



These are secrets.

You don't want:

Browser → Stripe secret key



Instead:

Browser



Backend



↓
Stripe API

The secret stays:

Backend environment



8. What about modern platforms like Firebase/Supabase?

Now here's the nuance. 🙄

You **can** have:

Browser
↓
Supabase
↓
Database



or:

Browser
↓
Firebase
↓
Database



And that can be a legitimate architecture.

Why?

Because these platforms don't simply give the browser your **database admin password**.

Instead, they provide controlled APIs and security mechanisms such as:

Authentication
+
Authorization rules
+
Row-level security
+
Restricted client credentials



For example, Supabase can expose a client-safe key while PostgreSQL **Row Level Security** determines what rows the user can access.

So the architecture becomes:

```
graph TD; Browser --> Supabase_API[Supabase API]; Supabase_API --> RLS_policies[RLS policies]; RLS_policies --> Database;
```



The database still isn't simply:

"Here's my PostgreSQL username and password, have fun." 😂

9. So is frontend → DB ALWAYS unsafe?

No.

That's the important nuance.

✗ Usually unsafe

```
graph TD; React --> PostgreSQL;
```



where you've exposed database credentials or unrestricted DB access.

✓ Can be safe

```
graph TD; React --> Managed_backend_service[Managed backend service]; Managed_backend_service --> Auth_authz_RLS[Auth + authorization + RLS]; Auth_authz_RLS --> Database;
```



Firebase/Supabase are examples of systems designed around this model.

10. Then what's the difference?

Think about **where your security rules live**.

Traditional architecture

Browser



Next.js Backend



Database



Backend handles:

Authentication

Authorization

Validation

Business logic

Secrets

Database access



Backend-as-a-service architecture

Browser



BaaS API



Security policies



Database



The managed platform handles much of that infrastructure.

11. And this connects directly to our Next.js authentication lesson

Remember our flow?

Browser



Server Action / Route Handler



Database



Why don't we do:

```
Browser
  ↓
Database
```



during login?

Because we'd be asking the browser to do something like:

```
"Hey database, give me the user whose email is X,
and here's the password."
```



Instead:

```
Browser
  |
  | email + password
  ▼
Next.js Server
  |
  | DB credentials stay here
  | password verification happens here
  | session creation happens here
  ▼
Database
```



That's a **trusted server boundary**.



The simplest way to remember it

Think of a restaurant. 🍴

```
Customer = Browser
Kitchen = Backend
Storage room = Database
```



You don't give every customer the keys to your storage room.

The customer says:

```
"I'd like paneer biryani."
```

The kitchen:

```
checks order
checks price
```



```
checks ingredients
prepares food
```

and accesses the storage room.

Similarly:

```
Browser
  ↓
"Give me my orders"
  ↓
Backend
  ↓
authenticate
authorize
validate
query DB
  ↓
Database
```



The browser **asks** for something.

The backend decides **whether and how** that request should be fulfilled.

Interview answer

If asked:

"Why can't the frontend directly access the database?"

A strong answer:

"A browser is an untrusted environment, so we shouldn't expose database credentials or unrestricted database access to client code. A backend provides a trusted server-side boundary where we can keep secrets, validate input, enforce authentication and authorization, execute business logic, and control database access. Some backend-as-a-service platforms allow controlled client-to-database access through APIs and policies such as Row Level Security, but that's different from exposing the database directly."

And there's one phrase I really want you to remember:

Frontend security controls UX; backend security controls access. 🔥

For example, hiding an **Admin** button in React is useful for UX, but the server must still verify `role === "ADMIN"` before performing the operation.





Awesome 🤗 Let's move to the **next important Auth.js concept**: how the session is **actually used to protect pages and APIs**.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

Google



OAuth/OIDC



Auth.js callback



↓
User/Account
↓
Session
↓
Cookie

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/  
├─ login/  
│   └─ page.tsx  
└─ dashboard/  
    └─ page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
</> TypeScript  
  
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}</h1>  
      <p>{session.user?.email}</p>  
    </div>
```

```
);  
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http  
  
GET /dashboard  
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript  
  
const session = await auth();
```



runs.

3. What does `auth()` do?

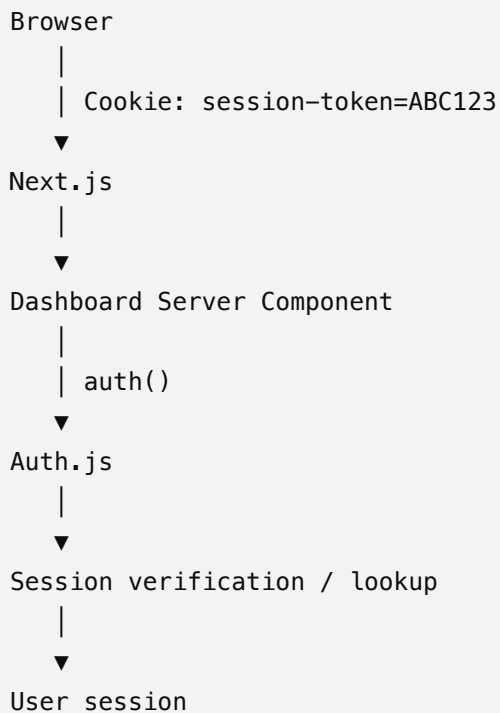
Remember where it came from:

```
</> TypeScript  
  
// auth.ts  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:



```
graph TD; Browser -->|Cookie: session-token=ABC123| Next.js; Next.js --> Dashboard[Dashboard Server Component]; Dashboard -->|auth()| Auth.js; Auth.js --> Session[Session verification / lookup]; Session --> User[User session];
```

Browser

Cookie: session-token=ABC123

Next.js

Dashboard Server Component

auth()

Auth.js

Session verification / lookup

User session

If the session is valid:

</> TypeScript

```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

</> TypeScript

```
const session = await auth();
```

returns something equivalent to:

</> TypeScript

```
null
```

So:

```
</> TypeScript

if (!session) {
  redirect("/login");
}
```



executes.

Next.js sends a redirect response.

Conceptually:

```
</> http

307 Temporary Redirect
Location: /login
```



The browser then requests:

```
</> http

GET /login
```



So:

```
User → /dashboard
      ↓
    auth()
      ↓
  no session
      ↓
redirect("/login")
      ↓
    /login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript
```



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript
```



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript
```



```
{session?.user.role === "ADMIN" && (  
  <button>Delete User</button>  
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



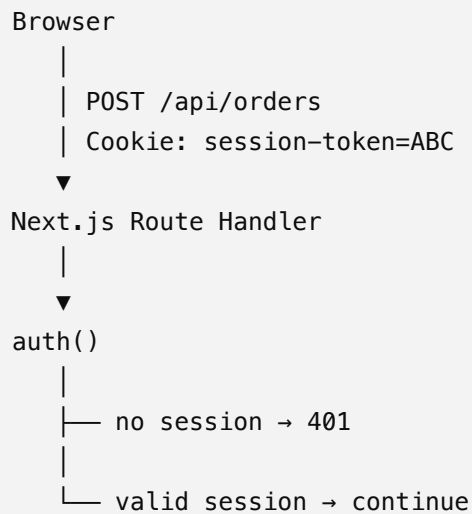
We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```



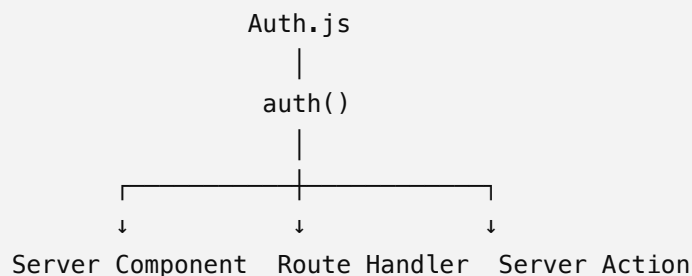
```
});  
}
```

Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

</> TypeScript

```
const session = await auth();
```

Route Handler

</> TypeScript

```
const session = await auth();
```

Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN

`signIn()`

↓

`OAuth`

↓

`callback`

↓

`session created`

2. READING SESSION

`auth()`

↓

`"Who is currently logged in?"`

3. PROTECTING ACTIONS

`auth()`

↓

`authenticate`

↓

`authorize`

↓

`perform operation`



That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

`</> TypeScript`



```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	Authorization / RBAC	Roles, permissions, admin/user access
7.10	Session Strategies	Database sessions vs JWT sessions

#	Topic	What you'll learn
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD
    Browser --> Login
    Login --> Authjs[Auth.js]
    Authjs --> Google[Google OAuth/OIDC]
    Google --> Callback
    Callback --> Exchange[Google token exchange]
    Exchange --> User[User + Account]
    User --> Adapter
    Adapter --> Database
    Database --> Session
    Session --> Cookie
    Cookie --> Request[Protected request]
    Request --> Auth[auth()]
    Auth --> Authentication
    Authentication --> Authorization
    Authorization --> Resource[Protected resource]
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.